

Blacphics System: Prioritized Architectural Refactor Plan

****Document Date:**** May 5, 2026

****Application:**** Blacphics POS/Inventory/Finance Management System

****Architecture:**** Django 6.0 + DRF + React + Vite + PostgreSQL

Executive Summary

This document provides a phased refactoring roadmap to improve the Blacphics system from a working prototype into a production-grade, maintainable, and scalable application. The plan is organized by priority (critical, high, medium, low) and estimated effort.

Critical Priority (Must Address Immediately)

1. Authentication & Authorization Layer

Priority: CRITICAL

Effort: 40-60 hours

Risk: Security vulnerability; unauthenticated API access

Current State

- No visible DRF permission classes configured
- Export endpoints explicitly allow `AllowAny` access
- Branch-level access control exists in data model but not enforced
- No user roles or teams defined

Refactor Steps

1. Add `DEFAULT\_PERMISSION\_CLASSES` to `REST\_FRAMEWORK` config

2. Implement role-based access control (RBAC):

- `super\_admin` (all branches)
- `branch\_manager` (own branch)
- `cashier` (own branch, read-only products/orders)
- `accountant` (finance, reporting)

3. Create `User.profile` extension model with role assignment

4. Implement branch-level queryset filtering:

```
```python
def get_queryset(self):
 user = self.request.user
 if user.is_superuser:
 return Model.objects.all()
 return Model.objects.filter(branch=user.profile.branch)
```
```

5. Secure export endpoints with permission checks

6. Add JWT token authentication (via `django-rest-framework-simplejwt`)

7. Implement API key authentication for integrations

Success Criteria

- All endpoints require authentication
- Branch-scoped queries automatically filter by user's branch
- Export endpoints protected
- Integration tests cover role-based access

2. Media Files Configuration

****Priority:**** CRITICAL

****Effort:**** 2-4 hours

****Risk:**** Image uploads fail; user data loss

Current State

- `Product.image` and `Expense.receipt` use `ImageField`
- No `MEDIA\_URL` / `MEDIA\_ROOT` in `settings.py`
- No image processing or storage strategy

Refactor Steps

1. Add to `settings.py`:

```
```python
MEDIA_URL = os.environ.get('MEDIA_URL', '/media/')
MEDIA_ROOT = os.environ.get('MEDIA_ROOT', BASE_DIR / 'media')
```
```

2. Configure URL routing in `Blacpicks/urls.py`:

```
```python
if settings.DEBUG:
 urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```
```

3. For production, integrate S3 storage (via `django-storages`):

```
```python
if not DEBUG:
 DEFAULT_FILE_STORAGE = 'storages.backends.s3boto3.S3Boto3Storage'
 AWS_STORAGE_BUCKET_NAME = os.environ.get('AWS_STORAGE_BUCKET_NAME')
```
```

4. Add image validation (size, format)

5. Document backup strategy for media files

Success Criteria

- Image uploads work in development
- Media URLs are accessible
- S3 integration tested (if using)

3. Order Number Generation - Concurrency Fix

Priority: CRITICAL

Effort: 8-12 hours

Risk: Duplicate order numbers under concurrent load

Current State

- `Order.generate_order_number()` uses Python-side logic with race condition
- Loops through all branch orders inefficiently
- Vulnerable under concurrent order creation

Refactor Steps

1. Create `OrderSequence` model:

```
python

class OrderSequence(models.Model):
    branch = models.OneToOneField(Branch, on_delete=models.CASCADE)
    next_number = models.BigIntegerField(default=1)

    class Meta:
        indexes = [models.Index(fields=['branch'])]
    ...
```

2. Replace Python logic with atomic database increment:

```
python

from django.db import transaction

from django.db.models import F

@transaction.atomic
def generate_order_number(branch_id):
    seq = OrderSequence.objects.select_for_update().get(branch_id=branch_id)
```

```

num = seq.next_number
seq.next_number = F('next_number') + 1
seq.save()
return f"ORD-{str(num).zfill(5)}"
...

```

3. Add migration to populate `OrderSequence` from existing orders
4. Remove old logic from `Order.generate\_order\_number()`
5. Add integration tests with concurrent order creation

Success Criteria

- No duplicate order numbers under concurrent load
- O(1) operation instead of O(n)
- Test passes with 100+ concurrent requests

4. Order Stock Logic - Deduplication & Clarity

Priority: CRITICAL

Effort: 16-20 hours

Risk: Stock deduction errors; double-counting

Current State

- Stock logic exists in three places:
 - `Order.complete\_order()`
 - `Order.reserve\_stock()` + `commit\_stock\_reservations()`
 - Signal handlers in `orders/signals.py` for quick sales and custom orders
- `complete\_order()` has a potential loop/indentation bug
- Mixing business logic across layers

Refactor Steps

1. Create `OrderStockService` in `orders/services.py`:

```

python

class OrderStockService:

    @staticmethod
    @transaction.atomic
    def reserve_stock(order, ttl_hours=4):

```

```

# Move Order.reserve_stock() logic here

@staticmethod
@transaction.atomic
def commit_stock(order):
# Move Order.commit_stock_reservations() logic here

@staticmethod
@transaction.atomic
def release_stock(order):
# Handle cancellation

@staticmethod
@transaction.atomic
def deduct_stock_quick_sale(order_item):
# Replace signal logic
...

```

2. Move signal-based deductions into service methods
3. Call service methods from models/viewsets (not signals)
4. Remove redundant logic from `Order` model methods
5. Add comprehensive unit tests for each scenario
6. Fix the `complete\_order()` loop bug

Success Criteria

- Stock logic consolidated in one service
- No signal-based side effects
- Unit tests cover quick sale, custom order, cancellation, partial refund
- All existing tests pass

High Priority (Implement in Phase 2)

5. Business Logic Service Layer

Priority: HIGH

Effort: 30-40 hours

Risk: Tightly coupled models; difficult to test

Current State

- Order payment logic in `Order` model
- Discount application in `Order.apply\_discount()`
- Finance calculations in `FinanceSelector` and views
- Mixing presentation with domain logic

Refactor Steps

1. Create `orders/services.py`:

```
```python
class OrderService:
 @staticmethod
 def apply_discount(order, amount, reason, approved_by):
 # Refactor Order.apply_discount()

 @staticmethod
 def record_payment(order, amount, method, payment_type):
 # Refactor add_payment logic

 @staticmethod
 def complete_order(order):
 # Orchestrate completion
```
```

2. Create `finance/services.py`:

```
```python
class FinanceService:
 @staticmethod
 def calculate_pl_report(branch_id, period):
```

```
Refactor calculate_pl_report
```

```
@staticmethod
```

```
def update_daily_snapshot(branch, date):
```

```
Generate DailyPLSnapshot
```

```
...
```

3. Refactor viewsets to call services instead of model methods

4. Keep selectors for read-only operations

5. Add service layer tests

### Success Criteria

- Models contain only schema and persistence logic
- Services contain business logic
- Views delegate to services
- All operations tested at service level

## 6. API Security & Validation

**Priority:** HIGH

**Effort:** 20-25 hours

**Risk:** Invalid/malicious data; data corruption

### Current State

- Basic DRF validation
- No request size limits
- No SQL injection protection (DRF handles this, but worth audit)
- Export endpoints lack validation
- No rate limiting

### Refactor Steps

1. Add `django-ratelimit` or `django-rest-framework-ratelimit`:

```
```python
```

```
@ratelimit(key='user', rate='100/h')
```

```
class OrderViewSet(viewsets.ModelViewSet):
```

```
    pass
```



```

2. Add request validation middleware:

```
```python
# Blacphics/middleware.py

class RequestValidationMiddleware:

    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        if len(request.body) > 10_000_000: # 10MB limit
            return JsonResponse({'error': 'Payload too large'}, status=413)
        return self.get_response(request)
```
```

3. Add input sanitization for text fields (escape HTML)

4. Validate file uploads (type, size)

5. Add CSRF middleware check

6. Document API security in code comments

### Success Criteria

- Rate limiting active
- Request size limits enforced
- All inputs validated and sanitized
- Security tests pass

## 7. API Error Handling & Standardization

**Priority:** HIGH

**Effort:** 12-16 hours

**Risk:** Inconsistent error responses; poor client UX

### Current State

- DRF default error responses (vary by serializer)
- No consistent error code structure
- No error logging
- Generic 500 errors don't aid debugging

## Refactor Steps

1. Create custom exception class:

```
```python
# common/exceptions.py

class APIException(Exception):

    def __init__(self, message, code, status_code=400, details=None):
        self.message = message

        self.code = code

        self.status_code = status_code

        self.details = details or {}
...

```

2. Create exception handler in `common/handlers.py`:

```
```python

def custom_exception_handler(exc, context):

 if isinstance(exc, APIException):

 return Response({

 'error': exc.message,

 'code': exc.code,

 'details': exc.details

 }, status=exc.status_code)

 return default_exception_handler(exc, context)
...

```

3. Register in `settings.py`:

```
```python

REST_FRAMEWORK = {

    'EXCEPTION_HANDLER': 'common.handlers.custom_exception_handler',

}
...

```

4. Add logging middleware to capture all errors
5. Use structured logging (JSON format)
6. Document error codes in API docs

Success Criteria

- All errors follow standard format
- Errors logged with context
- Clients can reliably parse error responses
- Integration tests verify error handling

8. Testing Infrastructure

Priority: HIGH

Effort: 25-35 hours

Risk: Regressions not caught; low code quality

Current State

- Test files exist but appear minimal
- No visible CI/CD pipeline
- No coverage thresholds

Refactor Steps

1. Set up `pytest-django`:

```
```bash
pip install pytest pytest-django pytest-cov
```
```

2. Create `pytest.ini`:

```
```ini
[pytest]
DJANGO_SETTINGS_MODULE = Blacphics.settings
testpaths = tests
addopts = --cov=orders --cov=products --cov=finance --cov-fail-under=70
```
```

3. Migrate existing tests to pytest conventions

4. Add test factories with `factory-boy`:

```
```python
class OrderFactory(factory.django.DjangoModelFactory):
 class Meta:
 model = Order
```
```

```
order_number = factory.Sequence(lambda n: f'ORD-{n:05d}')
```

```
status = 'pending'
```

```
'''
```

5. Add integration tests for critical paths

6. Set up GitHub Actions CI:

```
'''yaml
```

```
- name: Run tests
```

```
run: pytest --cov
```

```
'''
```

7. Require 70%+ coverage before merge

Success Criteria

- All tests run via pytest
- 70%+ code coverage
- CI runs on pull requests
- Critical paths tested (order creation, payment, stock)

9. Frontend/Backend Integration Setup

Priority: HIGH

Effort: 8-12 hours

Risk: CORS issues; environment misconfiguration

Current State

- React app exists separately in `frontend/`
- Django not serving React SPA
- CORS configured in Django, but frontend not tested with backend
- No environment variable coordination

Refactor Steps

1. Configure Django to serve React build in production:

```
'''python
```

```
# Blacphics/urls.py
```

```
if not DEBUG:
```

```
urlpatterns += [
```

```
path("", TemplateView.as_view(template_name='index.html')),  
path("", TemplateView.as_view(template_name='index.html')),  
]  
...
```

2. Add frontend build step to deployment:

```
```bash  
cd frontend && npm run build && cp -r dist/* ../static/
...
```

3. Verify CORS origins match frontend URL

4. Create `.env.example` for both backend and frontend

5. Document local development setup (run Django + Vite dev server)

6. Add E2E tests (Cypress/Playwright) for critical flows

### **Success Criteria**

- Frontend builds and deploys with backend
- CORS works in all environments
- Local dev setup documented and works
- E2E tests pass

## Medium Priority (Implement in Phase 3)

### 10. Query Performance Optimization

**Priority:** MEDIUM

**Effort:** 16-20 hours

**Risk:** N+1 queries; slow reports

#### Current State

- `common.selectors` uses `select\_related` and `prefetch\_related`
- Some optimization done, but no profiling
- Finance reports may be slow with large datasets
- No database indexing strategy

#### Refactor Steps

1. Install `django-debug-toolbar` for development:

```
```python
INSTALLED_APPS += ['debug_toolbar'] if DEBUG else []
MIDDLEWARE += ['debug_toolbar.middleware.DebugToolbarMiddleware'] if DEBUG else []
```
```

2. Profile order list endpoint and finance reports

3. Add database indexes:

```
```python
# orders/models.py
class Order(models.Model):
    class Meta:
        indexes = [
            models.Index(fields=['branch', '-created_at']),
            models.Index(fields=['status', '-created_at']),
            models.Index(fields=['payment_status']),
        ]
```
```

4. Optimize DailyPLSnapshot queries (consider materialized views or denormalization)
5. Add query monitoring in production (e.g., DataDog)
6. Document query optimization guidelines

## Success Criteria

- No N+1 queries in main endpoints
- Order list load time < 200ms
- Finance reports generate < 1s
- Indexes cover all common filters

## 11. Data Export & Reporting Enhancement

**Priority:** MEDIUM

**Effort:** 12-16 hours

**Risk:** Export failures; incomplete reports

### Current State

- PDF and Excel exports exist
- Limited to P&L; and orders
- No scheduled/background export jobs
- No export templates or customization

### Refactor Steps

1. Create export service:

```
```python
class ExportService:
    @staticmethod
    def export_orders_pdf(orders, branch_name):
        # Use reportlab

    @staticmethod
    def export_orders_excel(orders, branch_name):
        # Use openpyxl

    @staticmethod
    def export_inventory_report(branch_id, format='pdf'):
        pass
```
```

2. Add scheduled exports via Celery:

```

python

@periodic_task(run_every=crontab(hour=18, minute=0))

def export_daily_pl_report():

 # Email P&L; report to managers
 ...

```

3. Add export templates (customizable logos, headers)
4. Support for CSV exports
5. Add bulk export endpoint with async job handling

### Success Criteria

- Exports work reliably
- Scheduled exports run without errors
- Multiple formats supported (PDF, XLS, CSV)
- Export customization possible

## 12. Audit Logging & Compliance

**Priority:** MEDIUM

**Effort:** 14-18 hours

**Risk:** No accountability; compliance violations

### Current State

- `Order.discount\_approved\_by` tracks one action
- No comprehensive audit trail
- No user activity logging
- No data retention policies

### Refactor Steps

1. Create audit model:

```

python

class AuditLog(models.Model):

 user = models.ForeignKey(User, on_delete=models.SET_NULL, null=True)

 action = models.CharField(max_length=50) # 'create', 'update', 'delete'

 model_name = models.CharField(max_length=100)

 object_id = models.BigIntegerField()

```



```
old_values = models.JSONField(null=True)
new_values = models.JSONField(null=True)
timestamp = models.DateTimeField(auto_now_add=True)
ip_address = models.GenericIPAddressField()
...
```

2. Add audit middleware to track all changes:

```
```python
class AuditMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        # Capture state before, compare after
        ...
```

3. Add sensitive field masking (credit cards, passwords)

4. Implement data retention policy

5. Create audit report dashboard

6. Document GDPR compliance (if applicable)

Success Criteria

- All changes logged with user/timestamp
- Audit trail viewable in admin
- Sensitive data masked
- Retention policy enforced

13. Product & Variant Deduplication

Priority: MEDIUM

Effort: 6-8 hours

Risk: Code maintenance burden

Current State

- `ProductVariant.available\_quantity` and `stock\_status` defined twice
- Inconsistent logic duplication

Refactor Steps

1. Remove duplicate property definitions
2. Consolidate to single source
3. Add unit tests to verify behavior
4. Document product variant schema

Success Criteria

- No property duplication
- Tests pass

Low Priority (Nice-to-Have)

14. API Versioning

Priority: LOW

Effort: 8-10 hours

Current State

- No explicit versioning; all endpoints under `/api/`

Refactor Steps

1. Implement versioning via URL pattern or header:

```
```python
urlpatterns = [
 path('api/v1/', include('api.v1.urls')),
 path('api/v2/', include('api.v2.urls')),
]
```
```

2. Maintain backward compatibility
3. Document deprecation timeline

15. Notification System

Priority: LOW

Effort: 10-14 hours

Current State

- Branch manager email field exists, but usage unclear
- No notification framework

Refactor Steps

1. Create notification service (email, SMS, in-app)
2. Add event triggers (low stock, payment received, order completed)
3. Integrate with email backend (SendGrid, AWS SES)

16. Analytics Dashboard

Priority: LOW

Effort: 20-30 hours

Current State

- Finance reports exist
- No real-time dashboard or KPI tracking

Refactor Steps

1. Create analytics app
2. Add real-time metrics (orders today, revenue, avg order value)
3. Integrate with frontend dashboard

17. Documentation

Priority: LOW

Effort: 12-16 hours

Current State

- Code comments present but sparse
- No formal API documentation

Refactor Steps

1. Generate OpenAPI/Swagger docs via `drf-spectacular`
2. Document domain model and entity relationships
3. Create deployment guide
4. Add troubleshooting guide

Implementation Timeline

Phase 1 (Weeks 1-4): Critical

- Authentication & Authorization (60 hours)
- Media Files (4 hours)
- Order Number Generation (12 hours)
- Order Stock Logic (20 hours)
- **Total: ~96 hours (~2.5 weeks with 40 hrs/week)**

Phase 2 (Weeks 5-8): High Priority

- Business Logic Service Layer (40 hours)
- API Security & Validation (25 hours)
- Error Handling (16 hours)
- Testing Infrastructure (35 hours)
- Frontend/Backend Integration (12 hours)
- **Total: ~128 hours (~3 weeks)**

Phase 3 (Weeks 9-12): Medium Priority

- Query Performance (20 hours)
- Export & Reporting (16 hours)
- Audit Logging (18 hours)
- Product Deduplication (8 hours)
- **Total: ~62 hours (~1.5 weeks)**

Phase 4 (On-Demand): Low Priority

- API Versioning, Notifications, Analytics, Documentation
- **Total: ~56-60 hours**

Grand Total: ~242-246 hours (~6-7 weeks of development)

Success Metrics

- ☐ Zero critical security issues in OWASP Top 10
- ☐ 70%+ code coverage
- ☐ All endpoints return standardized error responses
- ☐ P99 API response time < 500ms
- ☐ No duplicate order numbers under load
- ☐ Stock never oversold
- ☐ All user actions audited
- ☐ Deployment automation in place
- ☐ Documentation complete and current

Risks & Mitigation

| Risk | Impact | Mitigation |
|-------------------------|---------------|--|
| Schema changes required | Data loss | Careful migration strategy; test backups |
| Scope creep | Timeline slip | Strict Phase adherence; defer Low priority |

Conclusion

This refactoring plan positions Blacphics as a robust, secure, and maintainable system. Prioritizing security and correctness (Phase 1) ensures the application is safe to use; subsequent phases improve developer experience and system performance.

****Recommendation:**** Begin Phase 1 immediately, aiming for completion within 3 weeks.